

Alunos(as): Beatriz Victoria Correa Ramos, Emanuely Carolina Roden, Djone Vitor Silvestre & Pedro Henrique
Turma: TDESS V2

Testes Não Funcionais (Performance, Carga e Estresse)

A. Introdução

Os testes não funcionais são fundamentais porque buscam validar como o sistema faz suas tarefas, focando no comportamento sob diferentes condições de exigência. O objetivo principal é encontrar o ponto exato da arquitetura (banco de dados, rede ou processamento) que está limitando o sistema, conhecido como gargalo de desempenho (bottleneck).

Para realizar esses testes, o QA utiliza ferramentas como JMeter, Gatling ou K6 para medir métricas como Throughput (vazão) e Latência (tempo de resposta). O processo envolve diferenciar a avaliação do tempo de resposta (performance), o comportamento no limite esperado (carga) e o comportamento além do limite máximo até a quebra do sistema (estresse).

B.



C. 5 Perguntas Desafiadoras

1) Qual é a diferença prática entre avaliar o tempo de resposta em testes de performance e o comportamento no limite esperado em testes de carga?

R = Enquanto a performance quer saber o quão **rápido** o sistema é, a carga quer saber se ele **aguenta** o tráfego máximo prometido sem falhar.

2)O que representam as métricas de performance "Throughput" e "Latência" na prática?

R = A **Latência** diz quão rápido um processo é; o **Throughput** diz quanto o sistema aguenta processar.

3)Por que a identificação do gargalo de desempenho (Bottleneck) é o objetivo principal, e não apenas a execução da ferramenta?

R = A identificação do gargalo (bottleneck) de desempenho é o objetivo principal, e não apenas a execução da ferramenta, porque **a ferramenta apenas fornece dados, enquanto a identificação permite a ação estratégica que resolve o problema real.**

4)Como ferramentas como JMeter ajudam a medir a escalabilidade do sistema?

R = O **Apache JMeter** é uma ferramenta open-source fundamental para medir a escalabilidade de um sistema, permitindo simular múltiplos usuários virtuais simultâneos para avaliar como a aplicação se comporta à medida que a carga aumenta.

5) Qual a diferença entre um teste de carga e um teste de estresse em relação ao ponto de quebra do sistema?

R = O teste de carga valida se o sistema suporta o volume esperado (uso normal/pico), operando dentro dos limites de desempenho. Já o teste de estresse busca ativamente o **ponto de quebra**, aplicando carga extrema para identificar quando e como o sistema falha.

D. Exemplos de Teste

- **Cenário:** Acesso simultâneo durante uma promoção.
 - **Como Testar:** Simular 10.000 cliques simultâneos usando JMeter.
 - **Objetivo:** Validar se o sistema suporta a carga sem degradar a performance.
- **Cenário:** Busca de produtos no banco de dados.
 - **Como Testar:** Realizar múltiplas consultas complexas e medir a latência.
 - **Objetivo:** Identificar se o processamento do banco é um gargalo de desempenho.
- **Cenário:** Teste de resistência máxima.
 - **Como Testar:** Aumentar o volume de dados além do limite máximo até a quebra do sistema.
 - **Objetivo:** Avaliar o comportamento do software em condições de estresse extremo.

Testes de Usabilidade e Acessibilidade Digital

Os **Testes de Usabilidade e Acessibilidade** existem **porque** de nada adianta um sistema ser rápido e seguro se as pessoas não conseguem entender como usá-lo ou se ele exclui parte do público. O objetivo principal é remover as barreiras que causam frustração, garantindo que o produto seja fácil de navegar e democrático para todos, independentemente de limitações físicas ou tecnológicas. Isso evita que a empresa perca usuários para a concorrência por causa de um design confuso ou inacessível.

Na prática, isso é feito através da observação e do uso de ferramentas específicas. No caso da usabilidade, recrutamos pessoas reais para tentar realizar tarefas no sistema enquanto observamos onde elas travam ou se perdem. Já na acessibilidade, o processo envolve usar tecnologias assistivas — como leitores de tela e comandos apenas por teclado — e seguir normas internacionais (como a WCAG) para conferir se cores, fontes e botões podem ser percebidos e operados por qualquer cidadão.

Visualização gráfica:



5 Questões:

1. Se um site é considerado "bonito" e "moderno", ele automaticamente tem uma boa usabilidade?

Resposta: Não. Beleza não é sinônimo de facilidade. Às vezes, um site muito moderno usa menus escondidos ou fontes muito claras que dificultam a leitura. A usabilidade foca em quão rápido e fácil você consegue resolver um problema. Se o design é lindo, mas você não acha o botão de "finalizar compra", a usabilidade é ruim.

2. Acessibilidade digital é algo que só ajuda pessoas com deficiência visual?

Resposta: De jeito nenhum. A acessibilidade ajuda muita gente: pessoas com braço engessado (limitação temporária), idosos que precisam de letras maiores, ou até você, quando tenta ler algo no celular sob a luz forte do sol (contraste). Ela é sobre criar um sistema que se adapta a qualquer condição humana, não apenas para um grupo específico.

3. Por que não basta apenas usar ferramentas automáticas para testar a acessibilidade de um site?

Resposta: Porque ferramentas não têm contexto. Um software pode dizer que uma imagem tem uma descrição (texto alternativo), mas só um humano consegue dizer se essa descrição faz sentido. As ferramentas automáticas pegam erros de código, mas os testes manuais e com usuários reais pegam erros de lógica e de experiência.

4. O que acontece se eu focar muito em Usabilidade, mas esquecer da Acessibilidade?

Resposta: Você cria um produto "VIP". Ele vai ser incrível para usuários padrão (que enxergam bem, usam mouse e têm boa internet), mas vai ser um muro intransponível para milhões de outras pessoas. Além de ser uma exclusão social, em muitos países (incluindo o Brasil), isso pode gerar problemas jurídicos e perda de mercado.

5. Se o usuário comete um erro ao usar o aplicativo, a culpa é sempre dele por não ter atenção?

Resposta: Na visão de UX, a culpa é quase sempre do sistema. Se o usuário errou, é porque o sistema não foi claro o suficiente ou não preveniu o erro. Um bom teste de usabilidade serve justamente para descobrir onde o design "enganou" o usuário e corrigir isso, tornando o caminho tão óbvio que o erro se torne quase impossível.

Exemplos:

1. Teste de Acessibilidade (Leitor de Tela)

Este teste garante que pessoas cegas consigam navegar no seu sistema.

- **Cenário:** O usuário está na página de "Carrinho de Compras" e precisa saber o valor total para finalizar o pedido.
- **Como Testar:** Feche os olhos (ou desligue o monitor) e tente navegar pela página usando apenas um software leitor de tela (como o NVDA ou TalkBack) e a tecla **Tab**.
- **Objetivo:** Verificar se o leitor anuncia claramente o preço total e se o botão "Finalizar Compra" é identificado corretamente pelo som, em vez de ser lido apenas como "botão" ou "imagem".

2. Teste de Usabilidade (Facilidade de Cadastro)

Este teste foca em saber se o seu fluxo de cadastro é intuitivo ou confuso.

- **Cenário:** Um novo usuário quer criar uma conta usando o celular.
- **Como Testar:** Peça para uma pessoa que nunca viu o app tentar se cadastrar. Observe (sem dar dicas!) se ela hesita em algum campo, se ela entende as mensagens de erro (ex: "senha muito curta") ou se ela desiste no meio do caminho.
- **Objetivo:** Identificar "pontos de atrito", como formulários longos demais ou botões que não parecem clicáveis, que fazem o usuário abandonar o aplicativo.

3. Teste de Acessibilidade (Contraste Visual)

Este teste ajuda pessoas com baixa visão ou quem está usando o celular sob sol forte.

- **Cenário:** O usuário está lendo os "Termos de Uso" ou uma mensagem de confirmação importante.
- **Como Testar:** Utilize uma ferramenta de inspeção de cores (como o *Color Contrast Analyzer*) para medir a diferença entre a cor do texto e a cor do fundo.
- **Objetivo:** Garantir que o contraste atinja o nível mínimo (geralmente **4.5:1**), para que o texto não "suma" no fundo, tornando a leitura confortável para qualquer pessoa.



Segurança: O QA como Defensor

O foco muda: o sistema precisa ser um **cofre**. Não testamos apenas funções, mas a resistência a ataques. Usamos o **OWASP Top 10** como guia para focar no que realmente causa invasões hoje.

Propósito

Transformar o sistema em um cofre.

Impacto

Evitar vazamento de dados e prejuízos financeiros.

Referência

OWASP Top 10 — ranking mundial de riscos.



Como Testamos a Segurança?

Regra de Ouro: "Nunca confie no que o usuário digita."

Não confiamos em inputs de usuários. Usamos o **SAST** para varrer o código atrás de erros e o **DAST** para simular ataques com o sistema no ar. O objetivo é **"sanitizar"** tudo que entra no sistema.



Exemplos de Testes de Segurança

Três exemplos reais de como testamos vulnerabilidades:

Cenário	Teste (Como)	Objetivo
Injeção SQL	Inserir ' OR 1=1 --	Barrar acesso sem senha.
XSS	Inserir <script>	Impedir scripts maliciosos.
Acesso	Tentar URL /admin	Bloquear perfis sem permissão.

No SQL Injection, tentamos enganar o login. No XSS, tentamos rodar um script num comentário. No Controle de Acesso, verificamos se um cliente comum consegue "pular" para a área administrativa.



Conclusão e Desafio

OWASP

O padrão ouro do mercado.

Risco Real

Inputs sem validação = porta aberta para hackers.

O software deve ser **rápido, seguro e inclusivo**.

DESAFIO FINAL

1. Performance, Carga e Estresse: Qual a diferença?

Imagine que você é dono de um restaurante:

Teste de Performance É cronometrar quanto tempo o garçom leva para trazer um prato quando o restaurante está tranquilo. O foco é a **velocidade**.

Teste de Carga : É ver como o serviço se comporta com todas as mesas ocupadas . O foco é saber se o sistema mantém a **estabilidade** sob a demanda prevista.

Teste de Estresse : É colocar uma fila de 500 pessoas na porta e ver o que acontece. O foco é descobrir **onde o sistema quebra** e como ele se recupera após o colapso.

4. As 10 Heurísticas de Nielsen

Pense nas Heurísticas como um "check-list de bom senso" criado por Jakob Nielsen para avaliar se um sistema é fácil de usar sem precisar de um manual. Elas orientam o design para ser intuitivo. Dois exemplos famosos são:

Visibilidade do status: O sistema deve sempre dizer o que está acontecendo (ex: uma barra de carregamento).

Saída de emergência: O usuário deve conseguir desfazer erros facilmente (ex: o botão "cancelar" ou "lixeira").

